

INF 345 — Fundamentals of DevOps

Lecture 3: Containers 101 — Why & How

Adil Akhmetov · Lesson 3



Recap — Lesson 2 (Git & GitHub)

- What command creates a new commit from staged changes? (`git commit`)
- How do you submit a lab in this course? (push to your GitHub Classroom repo)
- How fast do you get feedback on a lab submission? (Minutes — via GitHub Actions autograding)

Today's agenda

- ☐ Why containers exist (the problem they solve)
- ☐ Processes vs VMs vs containers
- ☐ Images, layers, and the Containerfile
- ☐ Podman vs Docker
- ☐ Live demo → straight into Lab 01

The scenario

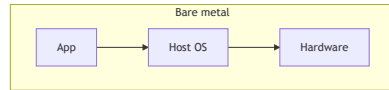
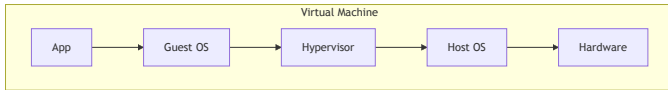
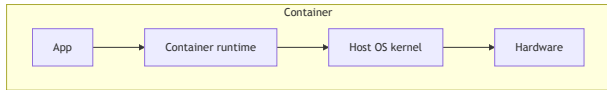
You write a Python app. It runs perfectly on your laptop. You hand it to a teammate — or deploy it to a server — and it crashes: wrong Python version, a missing system library, a config file that only exists on your machine.

Containers exist to make this scenario impossible.

Block 1

Processes, VMs, and Containers

Three ways to isolate an app



The key insight

- A **VM** virtualizes hardware — every VM ships a full guest OS. Heavy, slow to boot (minutes), strongly isolated.
- A **container** shares the host's kernel and isolates everything else (filesystem, processes, network) using Linux namespaces & cgroups. Light, boots in milliseconds, less isolated than a VM.

You don't need to understand namespaces/cgroups internals for this course — just this: a container is a normal process on your machine that *thinks* it has its own filesystem and network stack.

Quick check

Why do containers start so much faster than VMs?

No guest OS to boot — you're just starting a process against a kernel that's already running.

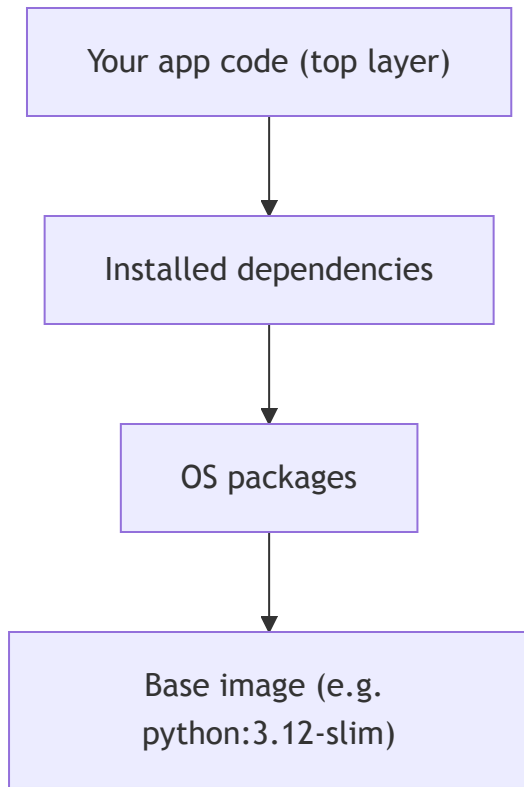
Block 2

Images, layers, and the Containerfile

Image vs container

- An **image** is a read-only template — like a class, or a recipe.
- A **container** is a running instance of that image — like an object, or the meal you actually cooked.
- You can run many containers from one image, each isolated from the others.

Images are built from layers



Anatomy of a Containerfile

```
FROM python:3.12-slim
RUN useradd -m appuser
WORKDIR /app
COPY app/requirements.txt .
RUN pip install -r requirements.txt
COPY app/ .
EXPOSE 8080
USER appuser
CMD ["python", "app.py"]
```

Click through: base image → create a non-root user → set up the working dir and install deps → copy in the app → expose the port → switch to the non-root user before the final command.

Notice the requirements file is copied and installed *before* the rest of the app — so editing app code doesn't invalidate the dependency-install layer's cache.

The non-root rule

- By default, a container's main process runs as **root** inside the container.
- "It's just a container" is not a good enough reason — a container escape or a mounted volume can turn that into real host access.
- Fix: create a non-root user and `USER` to it before the final `CMD` , like the last step of the Containerfile above.
- This is exactly the kind of thing the RHA DO188 lab checks for.

Block 3

Podman vs Docker, and today's lab

Podman vs Docker



VS



	Docker	Podman
Architecture	Client talks to a background daemon	Daemonless — runs directly as your user
Root by default	Yes (historically)	No — rootless by default
Commands	<code>docker build</code> , <code>docker run</code> , ...	<code>podman build</code> , <code>podman run</code> , ... (same syntax)
Compose	<code>docker compose</code>	<code>podman-compose</code> (drop-in)

Everything you write today — the Containerfile itself — is identical either way. DO188 uses Podman; you can practice with Docker too.

Live demo — build, run, verify

```
podman build -t demo-app .  
# → builds the image from your Containerfile  
podman run --rm -p 8080:8080 demo-app  
curl localhost:8080  
# → should return whatever your app responds with
```

This exact sequence — build, run, verify — is what you'll do inside the RHA DO188 lab. Nothing about the commands changes; RHA just gives you the environment to run them in.

By the end of this lesson, you should be able to

- ☐ Write a Containerfile from scratch for a simple app
- ☐ Explain why layer order affects build speed
- ☐ Build and run a container, and verify it responds correctly
- ☐ Explain why a container should not run as root

Before next lecture

- ☐ Start the RHA DO188 lab on rha.ole.redhat.com — this is what gets graded for this module
- ☐ See `labs/01-containers-podman/README.md` for details

Lab 01 is due Week 6 — you have time, but starting now means you're not rushing it later.

Next lecture

Images, layers, and multi-stage builds — making your images smaller and your builds faster.